

Linux:-

- Open Source, free operating System.
- Command line interface.

(GUI - Graphical User Interface uses icon) (Windows)
 (CLI - Command Line Interface uses text.)

Is Linux Really an OS?

- Linux itself = Kernel only.
- Full OS - Linux Kernel + GNU Utilities + Bash + Package Manager.

(GNU - Core Utilities, collection of free software system libraries).

(Bash - Default command line interpreter).

(Kernel - Core engine of OS).

Distribution of Linux:- (Different versions).

- Ubuntu - User friendly, for beginners.
- Debian - Stable and great for servers.
- Fedora - Cutting edge, sponsored by Red Hat.
- CentOS / Rocky / AlmaLinux - Enterprise Level.
- ArchLinux - Minimalist.
- Kali Linux - Cyber Security.

Kali Linux is a distribution used in Pen Test and digital forensics.

Metasploit, Burp Suite, Jack the Ripper are security tools.

Components of Linux:-

- USER
- Application
- Hardware
- Kernel - Core engine of OS.
- Shell/Bash - Command Line interpreter.
- File System Hierarchy Standard - Used in file organisation.

Types of Shell:-

- Bash
- C Shell (Csh)
- Korn (Ksh)
- Z Shell (Zsh)
- Enhanced C shell (tcsh).

Shell Commands:-

- echo - Used to print. $\Rightarrow$ echo "Hello"
- expr - Used for integer calculation $\Rightarrow$ expr 52 + 53
- clear - To clear commands.

Unix user - account

Password -

Installing Ubuntu:-

Installing Windows subsystem for Linux (Windows allow Linux to run in windows).

- Go to Search and search for windows features.
- Check the box of Virtual Machine Platform and Window Subsystem for Linux.
- In Command Prompt type `wsl -i` to install.

Basic Linux Commands:-

- pwd = Show directory to the working directory.
- uname -a = Tells username, OS - bit
- whoami = Username
- mkdir data = Used to create new directory
(Data is the name of directory).
- cd data = Change directory to (Data i.e) the already present directory).
- clear = Clean the terminal
- history = Used to see the used commands upto some extend.
- ls = List of files in the directory.
- vi = editor like visual Studio code ~~but~~ but for Linux

Installing of a application

Vim (Upgraded version of vi), Swicata, tcpdump.

sudo	{	apt	install	{	tcpdump
		apt-get			swicata
					vim

Sudo - tells admin is in control.

Checking installed application

`apt list -- installed.`

Uninstall a application:-

`sudo apt remove {tcpdump}`

vi or Vim Using

To edit = press "i" to insert.

To save and exit = press "esc" → ":wq"

To exit without saving = press "esc" → ":q!"

Note:- To copy = `ctrl + shift + c`

To paste = `ctrl + shift + v`

Commands Part-2:-

• `touch food.txt` - Create new empty file.

• `nano test.txt` - Editor in Linux

• `rm food.txt` - Remove or delete a file

→ `rmdir` - Used to remove empty directory

→ `rm *.txt` - Remove all ".txt" files

• `ls *.txt` - List of all files ends in ".txt"

• `cat hello.txt` - displays the content of the file without opening the file

Basic Linux Commands:-

- pwd = Home directory to the working directory.
- uname -a = Tells username, OS - bit
- whoami = Username
- mkdir data = Used to create new directory
(Data is the name of directory).
- cd data = Change directory to (Data i.e) the already present directory).
- clear = Clean the terminal
- history = Used to see the used commands upto some extend.
- ls = list of files in the directory.
- vi = editor like visual Studio code ~~but~~ for linux

Installing of a application

Vim (Upgraded version of vi), Suricata, tcpdump.

sudo { apt apt-get }	install	{ tcpdump suricata vim }
-------------------------------	---------	--------------------------------------

Sudo - tells admin is in control.

◦ `cp Shello.txt abc.txt` = copies the data from "Shello.txt" and paste to "abc.txt"

◦ `mv` = Move a file or rename a file.

→ `mv Shello.txt demo.txt` - Rename because no location changed only name changed.

→ `mv Shello.txt ~/Documents/` - Moves to other directory.

◦ `cat demo.txt >> hello.txt` = Append the content.

→ `history >> history.txt` = Create and append the "history" command values and save in "history.txt"

→ `echo "kindhi" >> history.txt` = Append "kindhi" in "history.txt"

Creating directory inside directory:-

`mkdir -p /home/kindhi/user/user1/widata`

Moving between directories:-

If system in directory `widata` in above example to move one directory behind

`cd ../`
for two directories → `cd ../../`

Finding Hidden Files:- (ls in detail)

- ls -l - Long listing format (permission, size etc)
- ls -s - Shows files size in blocks
- ls -t - Sorts by modification time
- ls -r - Reverse the sorting order
- ls -S - Sort by the file size (largest to smallest):

∴ Can be used as ls -ltr (combine the features).

ls -a - Finds ^(list) the hidden files.

Filtering Folders:-

- ls *.py = List all files in the directory that ends in py.
- ls time* = List all files in the directory that start in time

Hard Link, Soft Link and alias:-

Hard Link :- when you create a Hard Link, you do not duplicate the file content, another name created mapping the same storage.

* Changing of the files content, in original is also changes the content in new Hard Link file.

* If we remove one file other file still exist with same file info.

Soft Link - It's like desktop shortcut, It acts as a separate lightweight pointer file holding.

Hard Link and soft Link creation:-

Hard Link:-

In original.txt team2-data.txt

Soft Link:-

In -s original.txt new-data.txt

Alias:-

It is user defined shortcut replace a long complex command with shorter and easier-to-remember keyword.

```
$ alias kin="cd /home/kinoni/user/user/kiodek"
```

```
$ kin
```

- Only exist until the terminal close.
- To keep the alias permanent add in ~/.bashrc

Background Jobs, Nohup, and Process Monitoring.

Nohup - This command allows a process to keep running even after you log out or close your terminal window.

* Note: This process runs only on close of Terminal not shutting down of system.

For example:-

data_processing.py (File to be used).

\$ nohup python3 data_processing.py >> data_process.log &
{Enter twice}

\$top = Used to display running process and resource allocation

\$kill = Send specific command to running process, mostly to stop

\$ps = Process Status (Used in precise file running)

\$ps -aux

o a - Process for all users, not just the current one

o u - Show the user who own the process

and display in user format.

o x - Show processes not attached to a terminal (background stuff).

\$head = Display beginning of a file.

\$tail = Displays last 10 lines of a file

To specify:-

\$head or \$tail -10 data_process.log - Show 10 lines

To Kill a process:-

Step-1: `$ ps - aux`

Step-2: Copy Process Id.

Step-3: `$ kill -9 (165966) - Process Id.`

For simpler process.

Step-1: `$ ps - aux | grep data-processing.py` ↗ Used for filtering

Step-2: Copy Process Id.

Step-3: `$ kill -9 165966.`

Disk Management, Zip + Unzip + Tar / Untar

`$ df -h` = To show Size, used space, avail space

`$ du -sh script/` = Used to show ~~disk~~ used space (disk

usage)

`$ zip -r file.zip user/` = Used to zip (Compress) directory (user) with the name of file.zip.

`$ unzip file.zip` = Decompress file.zip.

\$tar -czvf sample.tar user/ = User to compress user ~~file~~ directory to name sample.tar

- c - Tell tar to create new archive file
- z - Compress archive file using gzip to reduce file size
- v - Displays the progress in the terminal (Verbose).
- f - Specifies the name of archive file.

\$tar -zxvf sample.tar = Used to decompress

-x = Tells tar to decompress.

\$free -m = Show RAM usage in mb.

\$free -g = Shows RAM usage in gb.

To clean cache uses the command:-

\$sudo sh -c "sync; echo 3 > /proc/sys/vm/drop-caches"

Word Count, Sort and Head.

Example:-

\$wc sample.txt

Output:- 3 9 61 sample.txt.

3 - Lines , 9 - words , 61 characters.

\$wc -w sample.txt = Counts only words

\$wc -l sample.txt = Counts only line.

\$wc -c sample.txt = Count only character.

\$ sort sample.txt = Sort in alphabetical order.

\$ sort -r sample.txt = Sorts in reverse order.

\$ sort -n sample.txt = Sort in ascending order (numbers).

\$ head = Displays first 10 line in the file.

grep:- Global regular expression.

Searches a ^{specific} file ~~on~~ and return all lines in the file containing a specific string.

! (Piping):-

Sends a standard output of one command as standard input to another command for further processing.

\$ grep WARNING *.txt = Searches for "WARNING" in all files ends in .txt.

~~\$ grep~~

\$ grep -i WARNING *.txt = Ignore case-sensitivity and searches for "Warning", "WARNING", "warning" etc.

\$ grep -v ERROR log1.txt = Leaves out error and gives all other lines.

DATE 19 06 2026

\$ grep WARNING *.txt | tail -f = Gives last 10 line which consist of WARNING.

Finding string in directories and its subdirectories.

\$ grep -r "Timeout" ~/logs

SSH and SCP

SSH - Secure Shell SCP - Command line utility used to (Secure log into and manage the Linux computer) | transfer file and directories between different host.

\$ ifconfig - Used to view and configure network interface.

\$ ssh 172.20.124.218 (Second mode)

\$ ssh ubuntu@xy (hostname)

Putty is used for SSH. It's a tool for SSH.

File transfer with SCP.

\$ scp /home/~~ubuntu~~^{kindni}/user/job_2022101.log ubuntu@hostname
/home/ubuntu/testcopy/joy.log.

WinSCP is used for SCP (SCP Tool)

Find:-

Find using name:-

```
$ find . -name log.txt
```

Searches from current directory.

```
$ find . -name *.txt
```

Searches all file ends in txt from current directory.

Find using file size:-

```
$ find . -type f -size +10M
```

Find using modified time.

```
$ find . -mtime -1
```

Shows all file modified within 1 day.

Finding empty files:-

```
$ find . -empty
```

Find and delete certain files.

```
$ find . -name "*.tmp" -delete
```


AWK:-

o `$awk '{print}' data.txt`

Shows all the data in file.

John 25 Developer

Asha 30 Designer

Ravi 28 Tester

o `$awk '{print $1}' data.txt`

Print first column.

o `$awk '{print $1, $3}' data.txt`

Print first and third column.

o `$awk '$2 > 27 {print $1, $3}' data.txt`

Print first and third column only that satisfies the condition.

o `$awk '{print "Name: " $1 " | Age: " $2 " | Role: " $3 "\n", $1, $2, $3}' data.txt`

Output:-

Name: John | Age: 25 | Role: Developer

Name: Asha | Age: 30 | Role: Designer

Name: Ravi | Age: 28 | Role: Tester

Change made :-

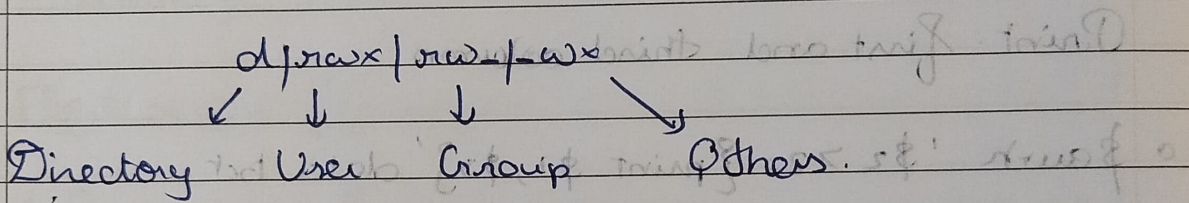
Permissions in Linux :-

- Read (r)
- Write (w)
- Execute (x)

Types of user :-

- User (u)
- Group (g)
- Other (o)

Example :-



Permission	Symbol	Value
------------	--------	-------

Read	'r'	4
Write	'w'	2
Execute	'x'	1

$6 = 4 + 2 \Rightarrow$ Read and ~~Execute~~ Write.

$5 = 4 + 1 \Rightarrow$ Read and Execute.

\$ chmod -R 777 /logs (Allowing all permission to /logs directory).

\$ chmod 777 new.txt (Allowing all permissions to the text file).

Change Ownership:-

- o `sudo chown new-owner filename`
- o `sudo chown Rob report.txt`

Changing owner and group.

- o `sudo chown Rob:developer report.txt`

Change group.

- o `sudo chgrp developer report.txt`

Change Ownership Recursively.

- o `sudo chown -R Rob:developer Project/`

Create New User:-

Delete a User

- o `sudo useradd john` / o `sudo userdel john`

Modify an Existing User:-

- o `sudo usermod -l john-new john` (Change Username).
- o `sudo usermod -aG sudo john` (Add user to group).

-a = append -G = Supplementary group -g = Primary group

- o `sudo usermod -d /home/newhome john` (Change directory)
- o `sudo usermod -L alice` (Locks the users account)

Shell Scripting:-

- Create a file named test.sh.

```
$ vi test.sh.
```

- Type.

```
#!/bin/bash  
echo "Hello World"
```

- Run the code.

```
$ sh test.sh.
```

Example 2:- Getting input.

```
#!/bin/bash  
echo "Enter your name:"  
read username  
echo "Welcome, $username!"
```

OUTPUT:-

```
Enter your name: xxx
```

```
Welcome, xxx!
```


Example 3:- If/else

```
#!/bin/bash
echo "Enter a number:"
read num
if [ $num -gt 10 ]; then
    echo "Number is greater than 10"
else
    echo "Number is 10 or less"
fi
```

Example 4:- Looping

```
#!/bin/bash
for i in 1 2 3 4 5
do
    echo "Looping... number $i"
done
```

Example 5:- Function

```
#!/bin/bash
say_hello () {
    echo "Hello from function!"
}
say_hello
```

Example 6:- Input arguments:-

```
#!/bin/bash
echo "Script name: $0"
echo "First Argument: $1"
echo "Second Argument: $2"
echo "Hello, $1 your role is $2."
```


• Save the file name as.

\$ sh arg-ex.sh kinthi csanalyst

↓ ↓
1st argument 2nd argument

Some other important commands:-

1. free -h = Displays RAM and swap memory usage.
2. less logfile.txt = Read a file without opening an editor.
3. sed 's/Linux/Ubuntu/' file.txt - Edit text from command line.
4. uniq fruits.txt = Shows only unique consecutive lines.
5. man grep = Shows the complete manual for a command.
6. whatis grep = Gives one-line explanation.
7. apropos network - Find command related to the topic.

Network commands:-

1. ping google.com - Check whether computer/server is reachable over the network.
2. curl https://example.com - Fetch data or interact with web API.
3. dig google.com - Display detailed DNS information.
4. nslookup - Find the IP address of a domain.
5. ss -tuln - Displays active connection and open ports.